

`DESIGN, DEVELOPMENT, MODELLING AND

CONTROL OF TRMS MIMO

A PROJECT REPORT

Submitted by

ABHISHEK MUNDRA [1171210089]

SHUBHANGI MAHESHWARI [1171210098]

LAASYA RAJAGOPAL [1171210117]

ASHWIN.K [1171210143]

Under the guidance of

Mr. Arockia Vijay Joseph, B.TECH, M.S

(Assistant professor (O.G), Department of Electronics and Instrumentation Engineering)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

In

ELECTRONOICS AND INSTRUMENTATION ENGINEERING

FACULTY OF ENGINEERING & TECHNOLOGY



SRM Nagar, Kattankulathur, Kancheepuram District

MAY 2016

SRM UNIVERSITY
(Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report titled “**DESIGN, DEVELOPMENT, MODELLING AND CONTROL OF TRMS MIMO**” is the bonafide work of **ASHWIN.K, LAASYA RAJAGOPAL, ABHISHEK MUNDRA and SHUBHANGI MAHESHWARI** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Mr. Arokia Vijay Joseph
Assistant Professor (O.G)
Electronic and Instrumentation
Engineering

SIGNATURE

Dr. Vimala Juliet
Head of Department
Electronic and Instrumentation
Engineering

Signature of Internal Examiner

Signature of External Examiner

Date:

ACKNOWLEDGEMENT

We take this opportunity to express a deep sense of gratitude to our Head of Department **Dr. Vimala Juliet** for her cordial support, valuable information and guidance, which helped us in completing this task through various stages.

We also take this opportunity to express our profound gratitude and deep regards to our guide **Mr. Arockia Vijay Joseph** for his exemplary guidance, monitoring and constant encouragement throughout the course of this project. The blessing, help and guidance given by him from time to time helped us a lot in the project and kept us on track.

We are also obliged to staff members of Electronics and Instrumentation Engineering department, for the valuable information provided by them in their respective fields. We are grateful for their cooperation during the period of our project.

Lastly we thank almighty and our friends, for their constant encouragement, without them this project would not have been possible.

ABSTRACT

Twin Rotor Multi Input Multi Output (TRMS) system is a laboratory test bench that mimics the dynamics of a helicopter. There is no widely agreed mathematical model for this system, as it is difficult to account complex aerodynamic effects. This has resulted in many dynamic models designed using different methods, but the models do not depict the original system dynamics. This project aims in design, development and modelling using parameter estimation(black box method) for TRMS. The TRMS modelling in the past were performed in white box and rarely grey box modelling environment. This project's aim is to make a black box model of the TRMS using parameter estimation. The main motivation for choosing black box analysis is that, whilst the system model is generated all the complex aerodynamic factors will already be encapsulated into the identification procedure and the model is perceived to be close in comparison with the original system. Finally PID controller is developed for the given model and implemented in the hardware.

TABLE OF CONTENTS

S.NO	CONTENTS	PAGE NO
1	Chapter 1- Introduction	1
	1.1 - Background Information	1
	1.2 Degree of freedom of Aeronautical System	5
	1.2.1 Yaw	5
	1.2.2 Pitch	6
	1.2.3 Roll	7
	1.3 Objectives	7
2	Chapter 2- Literature Survey	11
3	Chapter 3- Hardware Description	11
	3.1 – Mechanical Structure	12
	3.2 - Electronic Architecture	13
	3.3 – Uniqueness of existing TRMS	
4	Chapter 4 -System Identification	17
	4.1 – Classifications	18
	4.2 - Input to the System	20
	4.3 - Data Logging of the System Response	
	4.4 – Parameter Estimation for linear Module	20
	4.4.1- Auto Regressive with Exogenous Input(ARX) Model	20
	4.4.2 - Autoregressive Model Moving Average with Exogenous (ARMAX) Model	21
	4.5- Tuning Algorithm	22
	4.5.1-The least Square Algorithm	22

	4.6- Analysis and discussion	
	4.6.1- ARX analysis	22
	4.6.1.1- Original Output of the System	22
	4.6.1.2-First Order Response	23
	4.6.1.3-Second Order Analysis	23
	4.6.2-ARMAX Analysis	24
	4.6.2.1-Original output from IMU	25
	4.6.2.2-Second order Analysis	25
5	4.6.2.3-Third order Analysis	26
	4.6.2.4-Fourth order Analysis	26
	Chapter 5 – Controller Design	28
	5.1 PI Controller	29
6	5.2 PID Controller	30
	Chapter 6-Conclusions	32
	6.1- Observations	33
7	6.2-Recommendations	34
8	References	34
	Appendix	35

LIST OF FIGURES

S.NO	FIGURE NUMBER	PAGE NO
1.	1.1 - Dynamics test bench for cars	2
	1.2- Torque test bench for automobiles	2
	1.3-TRMS MIMO by Feedback Instruments	3
2.	1.4 - TRMS by Quanser	4
	1.5- Yaw degree of freedom	6
3	1.6-Pitch Degree of freedom	6
	1.7- Roll Degree of freedom	7
4.	3.1 - PICTURE	11
5.	3.2-Output Circuit	12
	3.3 - Data Acquisition Module	12
6.	3.3 - MPU 6050 used for feedback	13
	3.5 – Existing TRMS Design	14
	4.1- PRBS Generator	18
	4.2-PRBS Sequence	19
	4.3-Power density spectrum of PRBS	19
	4.4-SD card module circuit	20
	4.5-Original Output of the system	22
	4.6-First Order Response	23
	4.7-Second Order Response	24
	4.7- First order response of the system	24
	4.8-Second order response of the system	25
	4.9-Third order response of the system	26
	4.10] Fourth order response of the system	26
	5.1-Open loop Response	28
	5.2- PI Controller Response	29
	5.3-PID Controller response	30

CHAPTER 1

INTRODUCTION

Delay issues in control algorithms can be a big problem in aeronautical systems, as the gap for reaction time allowed is very less. Even a small error delay can cause entire system failure. To overcome this attempts have been made by engineers to develop test benches for UAVs and MAVs, with which the system

Twin Rotor Multiple Input Multiple Output system is a laboratory control test bench. In a TRMS there are two rotational actuators, two optical encoders and a control circuit. Both the rotors are placed orthogonal to each other to provide two different plane motion. The rotors provide yaw and pitch motion. The yaw motion is side to side movement of TRMS. The pitch actuation is the up and down actuation of the TRMS. This is a high end non linear system with a dynamic coupling between two planes due to propellers.

The TRMS is a common control problem used for validating new control techniques. To make its performance evaluation error free a more accurate model of the system is required. A lot of study is going on for improving the model of high end dynamic systems like TRMS. These studies can help us automating high end complex systems like ECMO which has a huge number of variables. This can help in developing aeronautical system stability improvement, production sector control problems.

There are many commercially available TRMS. Notable products are by Feedback Instruments [Fig. 1] and Quanser. Feedback Instruments is a company that produce academic purposes.



[Fig.1]TRMS MIMO by Feedback Instruments



[Fig. 2] TRMS by Quanser

The products provided by these companies are given with a constrained set of analysis. The product description of Feedback instruments gives a set of six studies into the system.

Where the proposed system in this project can be utilized for any number of control algorithms as the system can be easily to be changed with necessity.

After making a market study the team decided to build a self designed TRMS with a completely new modular design. In the present design there have been modifications made to the original system. The detailed mechanical structure has been discussed in the hardware description. These modifications made by the team have influenced the system dynamics.

There has been a lot of researches in the past that have been performed to model the TRMS. To have a noted few. Shih et al (2008) used Lagrangian approach to TRMS and analyzed propeller thrust and gravity compensation. Rahideh and Shaheed (2007) proposed two dynamic models of TRMS using Lagrangian and Newtonian propulsive forces of actuator and interfacing circuit. In the above discussed the Newtonian method was used to describe the 2 degrees of freedom system and the unpredictable dynamics were derived from the free body equations. Previously a black-box system identification technique was used by Ahmed et al(2000) to derive the dynamic model of TRMS. The paper also contains ideas of providing digital low pass and band stop(Butterworth filter) shaped input curves to obtain the specific vibration modes of TRMS test bed. Shih et al(2008) proposed a PD controller with a fuzzy PID control for set point tracking and stabilization. A robust controller was proposed Bayrak et al(2010) by assuming the TRMS was unknown(black box) and using a neural feed forward controller.

Following a complete set of literature study the team decided to come up with parameter estimation using ARMAX and ARX modeling. This study focus was chosen as the amount of practical implementation has yielded good results in all fields. The main reference material used in this project was “**Nonlinear System Identification using NARMAX**” by Prof. Stephen A Billings. The important reason for choosing this system identification technique is to make accurate black box model of the system and develop a more stable simulation data set. This project's completion is believed to make a huge impact on working with non linear systems and controlling them effectively.

LITERATURE SURVEY

Ahmed et al(2000) proposed a dynamic modeling of TRMS one degree of freedom of TRMS using black box system identification technique. The paper details about the effects cross coupling of one rotor over the other. A discrete time linear model of the system is identified and validated. The system's transfer function is used for designing a feed forward open loop control. As the open loop control of system is the first key for deriving more strong control laws for feedback systems, the system is resonated using shaped command input such as digital low pass and band stop(Butterworth) shaped inputs. Main observation derived in the system is that a low pass filter is shown to give reduced vibrations. The paper subscribes that a fourth order ARMAX analysis gave better results than the second order systems. The predictive capability of the model was identified to be matching and the oscillations were found to be matching. The model has been verified to frequency domain test, vibration tests and gave a proper output.

In contrast to the previous paper, A Rahideh and M H Shaheed have observed the system as grey box model in one and two DOF analysis. The back bone of their analysis was made by using Newtonian and Lagrangian approaches. The paper discusses modeling the system partially unknown with substituting a term for adjustments. They have performed the system in two phases. In the first pahse they have developed model for interfacing circuit and the propulsive forces that the motor can provide while having the Newtonian and Langrangian approaches common to both sub systems. The Lagrangian model proved to be better compared to the Newtonian model as the least square error average is less. This paper gave us a idea about using what physical methodology gave us better results.

In another interesting encounter, K.Triantafyllopoulos et al makes an interesting turn by estimating the parameters using ARX modeling technique and makes Baysian error reduction technique. This paper again concentrates on dynamic modeling of TRMS in hovering mode. This paper also discusses development of controllers apart from Baysian tuning. The output of the model with the original system is compared and found to give a good result. The Baysian formula used in the project makes a more accurate prediction of parameters. After successful completion of modeling the control algorithm with three variable PID had been developed. The

system response for different systems have been shown as it take more than 10 seconds to settle. The settling time for the setup is seen to be huge and the aim of our project is to reduce this time delay.

Intan Zaurah and Zainul Aman Lokaman(2010) have developed Neural network for modeling a TRMS only in horizontal motion. To compare the MLP-NN performance a parametric estimation using ARX modeling has been made. As explained by author every method has its own advantages and disadvantages, main problem being the models had very high settling time. This project aims in providing a platform for creating a highly accurate and immediate settling control algorithms.

HARDWARE DESCRIPTION

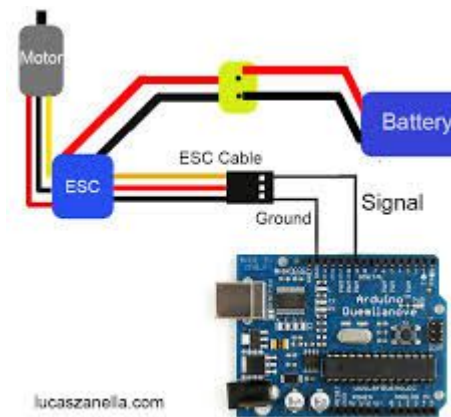
In this section we will see the hardware attributes of constructed TRMS. The TRMS built has been constructed by taking ideas from previous designs in terms of mechanical structure and electronic architecture. Every aspect of TRMS has been done with complete analysis of existing designs and the best features among them has been included into the built model. Let us discuss all the aspects in detail,

Mechanical structure :

After a complete analysis of existing models of TRMS the team came up with a unique design of TRMS. The mechanical structure basically consists of a heavy base made of mild steel material. This base material is fit to dampen the vibrations generated in the actuator in the top. This base is given a vertical beam that originated from the mild steel base, and provides a base for holding the upper setup. In the top the actual TRMS setup is given with a beam mounted on a prismatic joint with the horizontal axis. The rotational joint is given a mount for giving the TRMS a vertical rotational freedom. Thus this complete setup is now given a yaw and pitch rotational freedom with the base beam. The beam apart from original geometry, is given a counter balance with is fit with the pitch rotational axis to maintain its null position when not actuated. On either ends of the TRMS beam an actuator is provided and this forms the complete mechanical design of TRMS. BLDC motors are used as actuators.

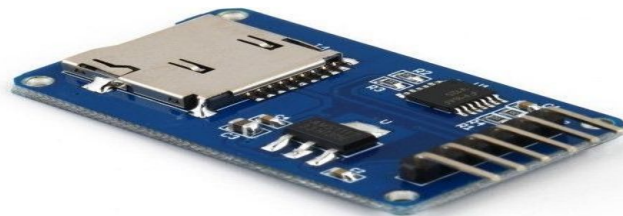
ELECTRONIC ARCHITECTURE

The electronic system mainly consists of a power source which drives the entire circuitry. The TRMS electronic hardware can be classified as input, processing and output. The TRMS output system consists of two BLDC motor which are fit to either end in perpendicular fashion. In this design due to inclusion of BLDC motor, a separate ESC circuitry is concatenated with the usual power source. This forms the output electronic system.



Output circuit

The processing part of the system is taken care of by a microcontroller. The microcontroller used in this project is arduino Mega. All the instruction to the ESC and receiving of input from feedback circuitry is performed using Arduino Mega. The Mega version was chosen for the project as it was necessary to perform both processing and data acquisition. Data acquisition was performed by using CALTAX micro SD card Adapter. By this system the team developed a cheap and effective Data acquisition System.



Data acquisition module

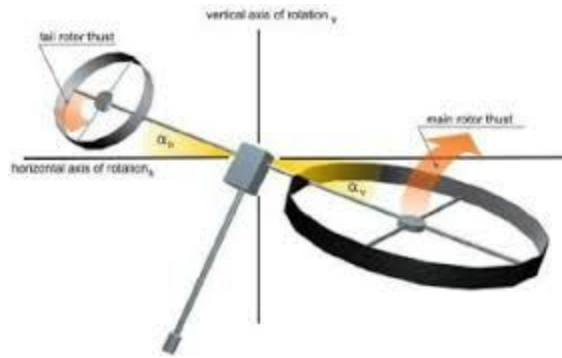
In the previous designs of TRMS the system used potentiometric feedback which made servicing and working difficult. But in the current design, the feedback system is changed to an Inertial Measurement Unit. In this project we used MPU 6050 as the feedback element. The main advantage is that it has a very cheap installation cost and it develops no effects on the system freedom. This improves the system's quality of estimation. It transmits 1000 data strings updating accelero-gyro and orientation values per second. It transmits the data using I2C communication port.



MPU 6050 used for feedback

UNIQUENESS OF CONSTRUCTED TRMS:

Twin motor multi input multi output system (TRMS) is a laboratory set up resembling a helicopter which is designed for conducting control experiments. The mechanical part of the existing system model includes two rotors on a beam with a counter balance. The main rotor produces vertical thrust and rotates the beam around pitch axis. The other rotor, called as the tail rotor, is mounted on the opposite side of the beam, perpendicular to the main rotor, to obtain movement on the yaw axis.



Existing TRMS Design

In construction of our TRMS model there have been made a few replacements and adjustments done when compared to the existing model. By doing so a more accurate analysis can be done. It includes using of brushless DC motors instead of regular ones and placing the counter balance weight in alignment to the beam which usually is protruded out.

As mentioned earlier in this developed TRMS we use a brushless DC motor over the existing one. The most obvious advantage of a brushless motor is its lack of brushes and physical commutator. An electronic controller replaces the brush, commutator assembly of the brushed DC motor. This difference means that there are many fewer parts that can wear out or break and need to be replaced than in a brushed motor. A BLDC motor tends to be more reliable, lasts longer and be more efficient. It also offers several advantages including high torque to weight ratio, more torque per watt, increased reliability, reduced noise, longer lifetime etc. With no windings on the rotor, they are not subjected to centrifugal forces and because the windings are supported by housing they can be cooled by conduction. This in turn means that the motor's internals can be entirely enclosed and protected from dirt and other foreign matter.

The other change found in the developed model is the position or placement of the counter weight. In the existing system we can observe the counter weight hanging out on the joined beam which is used for balancing the angular momentum. It is used to determine the

stable equilibrium position. The system is balanced in such a way, that when the motors are switched off the main rotor end of the beam is lowered down due to its weight. However when the counter weight is being inbuilt with the axis of the beam the results improved considerably.

Apart from the above mentioned changes a new work bench model has been designed in our laboratory. This model enables us to have a complete thorough study of the pitch axis. In the existing model the beam of TRMS is bottom fixed and it has a complete access of the yaw motion and whereas limits the pitch motion to about 270° only. This limitation is because it is fixed. So a new construction is made which is top fixed and has a structure of "H". This model is hung by a bearing which is supported by wooden boxes on either end. By this design the pitch motion can attain a complete angle of 360° . And as its hung freely about its axis there are no limitations for yaw motion as well. Also by this design a further research and test can be performed on the TRMS for roll motion. This newly developed design has been proven useful not only TRMS but other similar control system models. It can be termed as a universal work bench in a way which can be used in laboratories for experimental purposes in academics.

SYSTEM IDENTIFICATION:

A mathematical model of a system can be used to emulate the system, predict the system response for given inputs, and investigate different design scenarios. Obtaining systematic information from experimental observations is a key aspect of any scientific work. The obtained information leads to the formulation of a model of the system under consideration. However, these objectives can only be achieved if the model of the system is known. For this purpose System Identification is done. SYSTEM IDENTIFICATION is a method of measuring the mathematical description of a system by processing the observed inputs and outputs of the system. It is the complement of simulation problem.

The field of **system identification** uses statistical methods to build mathematical models of dynamical systems from measured data. System identification also includes the optimal design of experiments for efficiently generating informative data for fitting such models as well as model reduction. The resulting dynamic mathematical model from system identification tools can be further used to perform simulation and prediction, which involves a wide range of applications. As well the advantages of system identification are many : it is applicable to all systems, it is often quick, and can be made to track changes in the system.

System identification is mainly used in the following three areas:

- **Modeling and Simulation**—The modeling of industry equipment can be used for fault diagnosis and machine monitoring. In biology and medicine, modeling and simulation of human organs can facilitate diagnosis of diseases.
- **Control Design**—In automatic controller design, obtaining dynamic properties of control objects is prerequisite for formulating effective control strategies and achieving high control performance.
- **Prediction**—System identification can be used to aid the prediction of commodities, prices, weather patterns, and expectation of power load.

System identification is usually carried out in an iterative process. The whole process includes:

- Performing experiment to collect real input and output data from the physical model to be identified.
- Selecting suitable mathematical model with unknown parameters to represent the physical system you want to identification.
- Estimating unknown parameters defined in selected mathematical model with system identification tool.
- Validating the estimated model if it fit real input and output data well.

The different methods of system identification are:

- White-box identification
 - estimate parameters of a physical model from data
 - Example: aircraft flight model
- Gray-box identification
 - given generic model structure estimate parameters from data
 - Example: neural network model of an engine
- Black-box identification
 - determine model structure and estimate parameters from data
 - Example: security pricing models for stock market

CLASSIFICATIONS

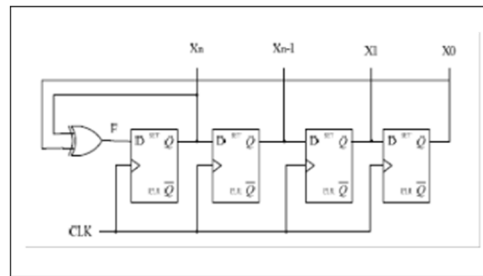
As mentioned before models play a central role in system analysis, design, and simulation for which system identification is a technique that can be used to infer and construct system models from experimental data. Systems can be linear or non-linear, time-invariant or time-variant, continuous-time or discrete-time, lumped-parameter or distributed-parameter based on which classification is made.

Amongst these model types, the autoregressive moving average with exogenous input (ARMAX) model, including subsets of AR (autoregressive), ARMA (autoregressive moving average), and ARX (autoregressive with exogenous input) models are the most commonly used representations for system identification. The non linear autoregressive moving average with exogenous input (NARMAX) models are, where the non linear AR and ARX models are the most popular representations for nonlinear dynamic system identification in discrete-time domain,

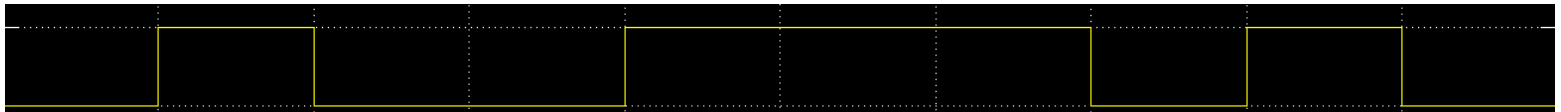
For our designed TRMS model we have considered system identification through linear models such as ARMAX and subsets like ARX and ARMA modeling. Linear models are the most frequently used representation for systems whose inherent structure is linear by nature and where the system obeys the superposition principle, or whose inherent behavior can be characterized by a linear approximation close to a system operating point.

INPUT TO THE SYSTEM:

The System Identification tool used in this project has been performed for black box modeling. The main objective of the modeling is to observe the system's dynamics in all frequencies. One of the most widely used signal that is used to observe the system in all frequencies is the PRBS signal. The PRBS (Pseudo Random Binary Signal) is a set of repetitive bits that follow a unique fashion. The circuit follows as the first bit is derived as the exclusive or combination of first and the last bit.

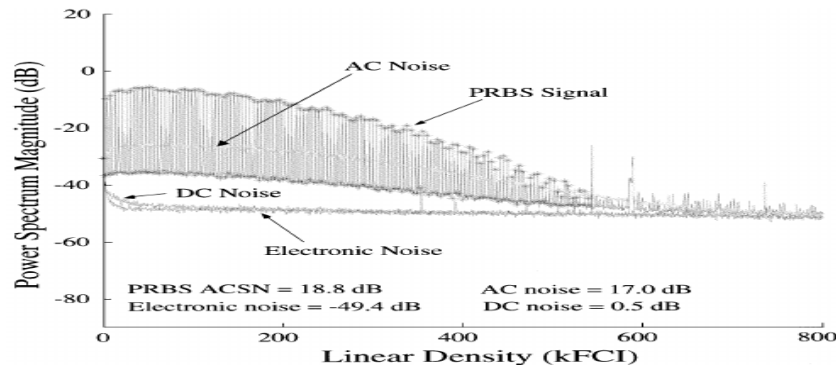


PRBS Generator



PRBS Sequence

The above sequence is the PRBS signal with 3 bit seed. The main advantage being the highly effective frequency response, the system identification becomes more rugged and reliable.



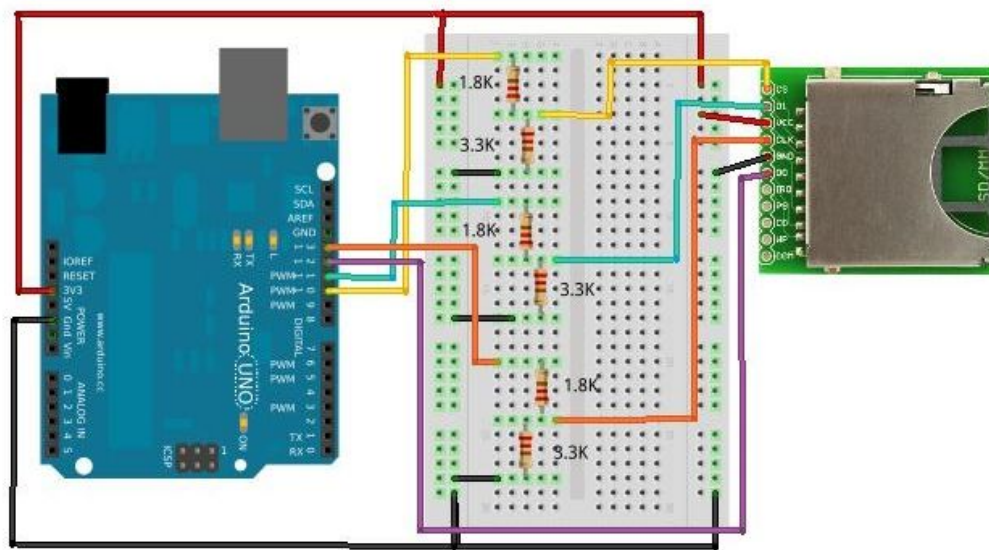
Power density spectrum of PRBS

As observed from the graph the system's power peaks are seen to be high compared to other noise signals. The team generated PRBS signal using Arduino using a basic logic and gave it as an input to the system in its maximum ranges of linearity.

The ARMAX and ARX analysis after the acquisition of data using the PRBS as the actuating signal, the mathematical modeling of the system is begun. The input output data is sorted into the matrix format as shown above and the least square estimate is calculated. The system's equation for three degree of freedoms were calculated.

DATA LOGGING OF SYSTEM RESPONSE:

The system is made to actuate according to the PRBS input signal and the response of the system is recorded into the data circuitry. The system contains a SD card module which is compatible with Arduino. The data logger is commanded to log a value of data, when the complete bit period is absorbed by the system. The SD card module is provided with a SD card in which the system data are logged. The following is the block diagram of the system data logging circuit.



SD card module circuit

PARAMETER ESTIMATION FOR LINEAR MODEL :

Auto Regressive Moving Average with Exogenous Input(ARMAX) Model :

The general model structure of linear finite order system can be formulated as

$$A(z^{-1})y(k) = \frac{B(z^{-1})}{F(z^{-1})}u(k) + \frac{C(z^{-1})}{D(z^{-1})}\xi(k)$$

where ,

$$A(z^{-1}) = 1 + a_1 z^{-1} + \dots + a_{na} z^{-1}$$

$$B(z^{-1}) = 1 + b_1 z^{-1} + \dots + b_{nb} z^{-1}$$

$$C(z^{-1}) = 1 + c_1 z^{-1} + \dots + c_{nc} z^{-1}$$

$$D(z^{-1}) = 1 + d_1 z^{-1} + \dots + d_{nd} z^{-1}$$

$$F(z^{-1}) = 1 + f_1 z^{-1} + \dots + f_{nf} z^{-1}$$

$\{y(k)\}$ and $\{u(k)\}$ ($k=1,2,\dots$) are the system input and output signals respectively

na, nb, nc, nd, nf are non negative integers indicating the model orders

$\{\xi(k)\}$ is noise sequence which is assumed to be independently and identically distributed(i.i.d) with zero mean and finite variance.

Autoregressive Model with Exogenous (ARX) Model:

Setting $C(z^{-1}) = D(z^{-1}) = F(z^{-1}) = 1$ gives the ARX (autoregressive with exogenous input) model.

$$y(k) + a_1 y(k-1) + \dots + a_{na} y(k-na) = b_1 u(k-1) + \dots + b_{nb} u(k-nb) + \xi(k)$$

TUNING ALGORITHM :

The least Square Algorithm:

The method of least square is a standard approach in regression analysis to the approximate solution of over determined systems, i.e., sets of equations in which there are more equations than unknowns. “Least squares” means that overall solution minimizes the sum of the squares of the errors made in the results of every single equation. The least squares method is usually credited to Carl Friedrich Gauss but it was first published by Adrien-Marie Legendre.

The popularity of the ARX model stems from the fact that the model parameters can be directly estimated using a least squares (LS) algorithm.

Let $p=1+\max(n_a, n_b)$, and

$$y = [y(p), y(p+1), \dots, y(N)]^T$$

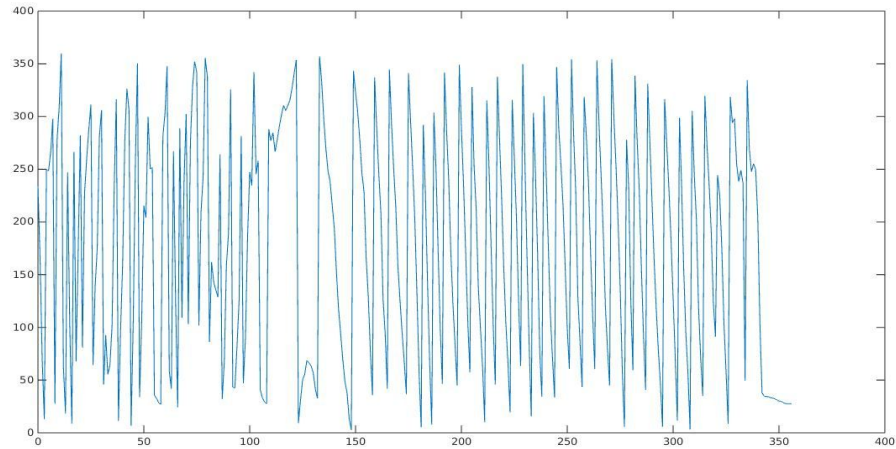
$$\xi = [\xi(p), \xi(p+1), \dots, \xi(N)]^T$$

$$\theta = [a_1, a_2, \dots, a_{n_a}, b_1, b_2, \dots, b_{n_b}]^T$$

ANALYSIS AND DISCUSSIONS:

ARMAX analysis :-

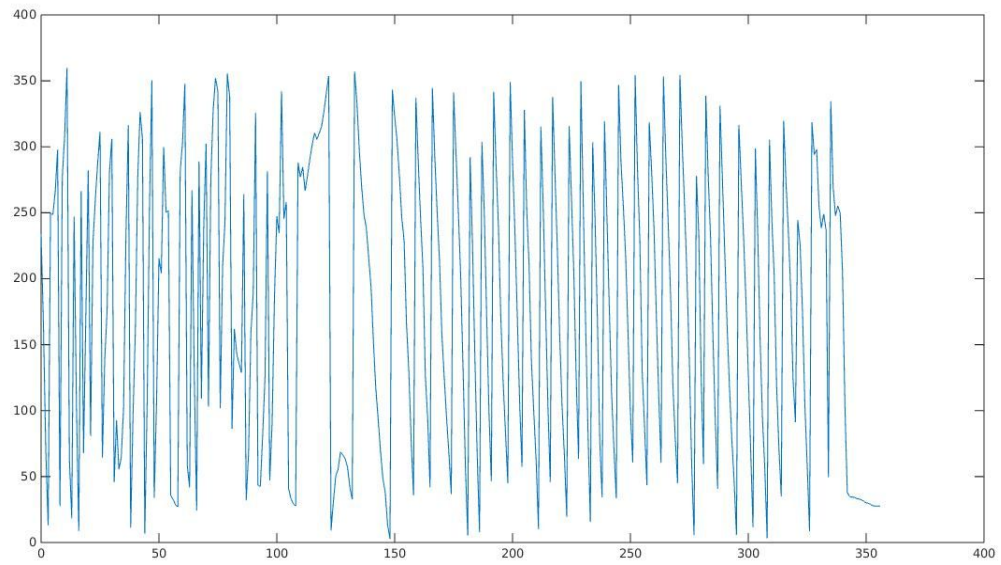
First Order Analysis :



First order response of the system

It can be observed from the first order plot the system settles with the original system completely without any error. This is possible because the system is a partially linear and also the noise involved is relatively less. This was referred to Mr.Stephen.A,Billings.This gave us a more accurate model of first order system. This first order settling of system clearly explains the power of ARMAX model, using which all control algorithms were designed.

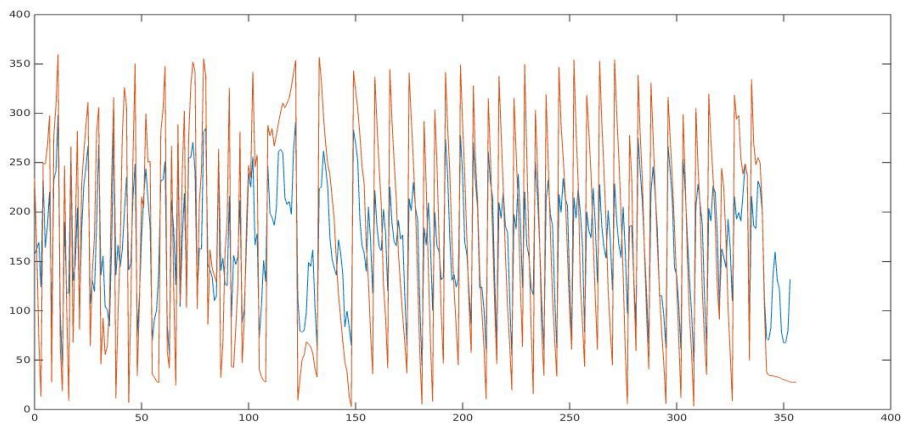
Second Order Analysis :



In the second order analysis of the system, the graph although interlinks with the original system, the system when observed in MATLAB gave a continuous small deviation error at each point. This can be inferred as the system when simple and viewed in details, it reduces the noise tolerance.

Such small deviation shows us that ARMAX models for lower order system when tried to observe at higher order can make the estimated model to become less resistance to noise and the consecutive analysis by incrementing the order can make the noise margin wider.

Fourth Order Analysis :



Fourth order analysis

It can be clearly noted as the system increases the error tolerance of the system becomes relatively difficult. This can result in improper model estimation. So one of the main result of this project has been the importance of analyzing a system in different orders and observe its closeness and improve the system model accuracy. A detailed report on this is given in the conclusion of the report.

CONTROLLER DESIGN:

Controllers are used to automatically adjust some variable to hold the measurement (or process variable) at the set-point. The set-point is where you would like the measurement to be. Error is defined as the difference between set-point and measurement.

$$(\text{error}) = (\text{set-point}) - (\text{measurement})$$

The variable being adjusted is called the manipulated variable which usually is equal to the output of the controller.

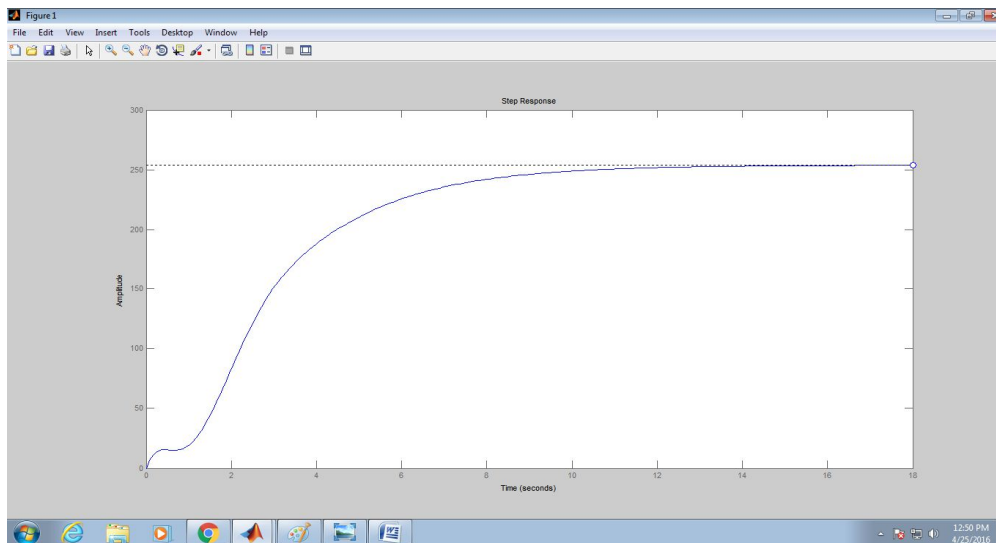
The discrete time domain transfer function obtained by the applying formula is

$$\frac{19.2z^2 + 52.75z + 32.68}{z^3 - 0.5707z^2 + 0.0445z - 0.0617}$$

When converted to the continuous time domain system, we get the transfer function as,

$$\frac{98.98s^2 - 168.2s + 468.7}{s^3 + 2.785s^2 + 5.284s + 1.846}$$

On giving a step unit as input, the open loop response is observed as,



From the above figure we can conclude that the system is unstable and exhibits non linear characteristics.

PI controller

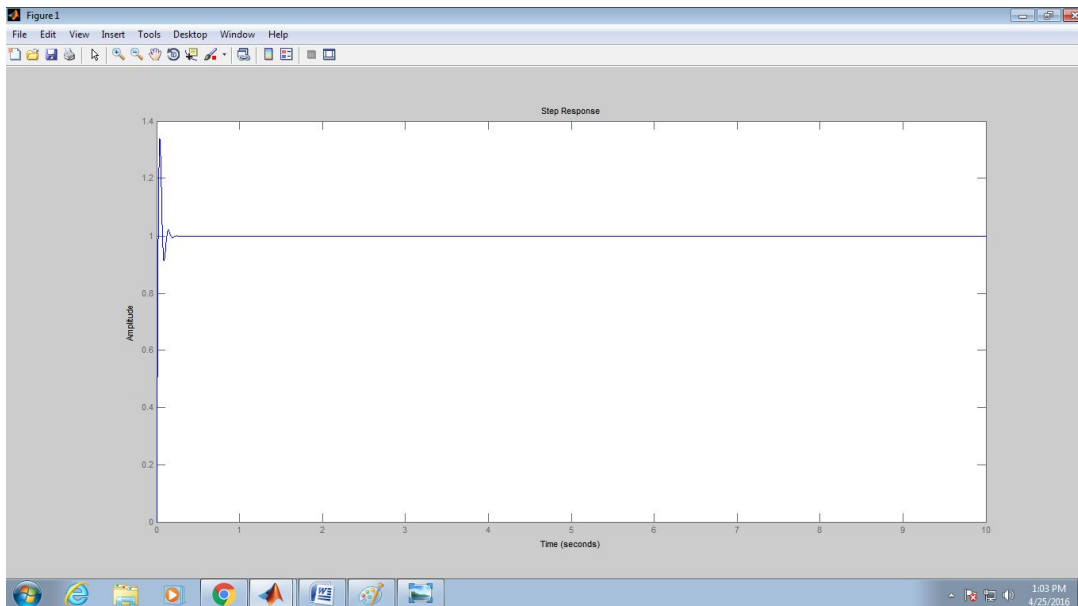
To make the system more linear and stable only proportional controller is not enough because with proportional band, the controller output is proportional to the error or a change in measurement (depending on the controller).

$$(\text{controller output}) = (\text{error}) * 100 / (\text{proportional band})$$

With a proportional controller offset (deviation from set-point) is present. Increasing the controller gain will make the loop go unstable. Integral action was included in controllers to eliminate this offset.

So we decided to design a PI controller as with integral action, the controller output is proportional to the amount of time the error is present. Integral action eliminates offset.

$$\text{CONTROLLER OUTPUT} = (1/\text{INTEGRAL}) (\text{Integral of } e(t) \, dt)$$



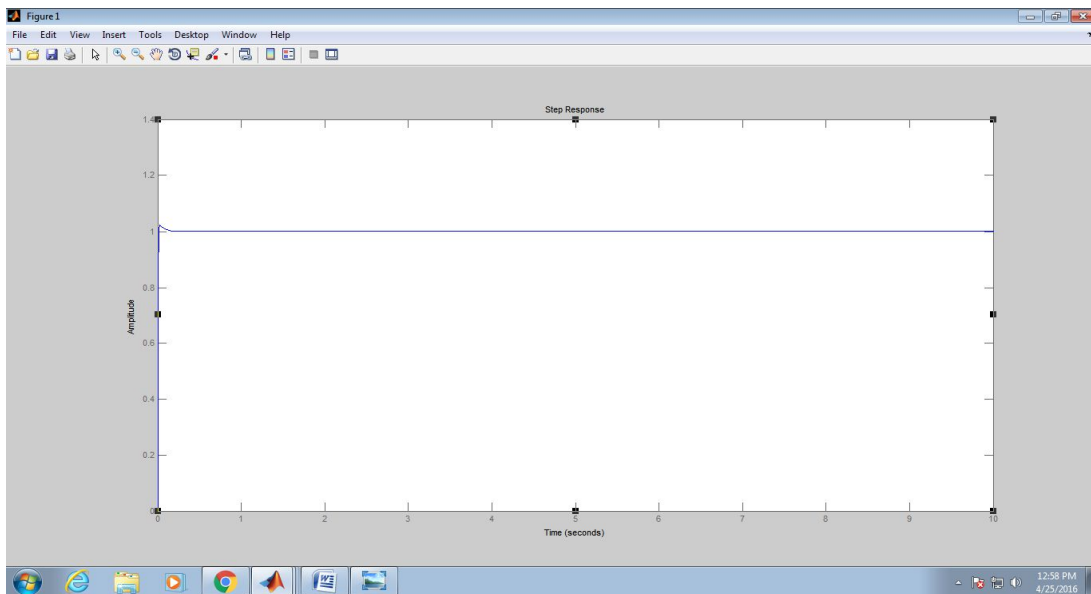
The response is showing oscillatory response in the starting and can be stabilized some by adding derivative action.

PID controller

With derivative action, the controller output is proportional to the rate of change of the measurement or error. The controller output is calculated by the rate of change of the measurement with time.

$$\text{CONTROLLER OUTPUT} = \text{DERIVATIVE} \frac{dm}{dt}$$

Where m is the measurement at time t.



Derivative action can stabilize loops since it adds phase lead. Generally, if you use derivative action, more controller gain and reset can be used.

With a PID controller the amplitude ratio now has a dip near the center of the frequency response. Integral action gives the controller high gain at low frequencies, and derivative action causes the gain to start rising after the "dip". At higher frequencies the filter on derivative action limits the derivative action. At very high frequencies (above 314 radians/time; the Nyquist frequency) the controller phase and amplitude ratio increase and decrease quite a bit because of discrete sampling. If the controller had no filter the controller amplitude ratio would steadily increase at high frequencies up to the Nyquist frequency (1/2 the sampling frequency). The controller phase now has a hump due to the derivative lead action and filtering.

The time response is less oscillatory than with the PI controller. Derivative action has helped stabilize the loop.

STATE SPACE MODEL:

State-space models are models that use state variables to describe a system by a set of first-order differential or difference equations, rather than by one or more nth-order differential or difference equations. State variables $x(t)$ can be reconstructed from the measured input-output data, but are not themselves measured during an experiment. A state-space model is just a structured form or representation of the differential equations for a system.

State-space models are useful in many situations:

- Linearization of non-linear models
- Calculation of time-responses — both analytically and numerically
- Using simulation tools: MATLAB, LabVIEW, Octave, and Scilab have simulation functions that assumes state-space models.
- Analysis of dynamic systems, e.g. stability analysis

- Analysis and design of advanced controllers and estimators: Controllability and observability analysis; Design of LQ optimal controllers, Model-based predictive control, and Feedback linearization control; Design of state estimators (Kalman filters).

In this project for the developed system PID control has been implemented and their required parameters have been generated. The next operation is to apply the state space modeling techniques and analyze the system. To start with we check if the system is completely observable and controllable. This can be done either manually or can be done through Matlab. After this a suitable controller is designed for the converted state space parameters and it is simulated.

Controllability:

Complete state controllability (or simply controllability if no other context is given) describes the ability of an external input to move the internal state of a system from any initial state to any other final state in a finite time interval.

Controllability Matrix

$$Co = [B \ AB \ A^2B \ \dots \ A^{n-1}B]$$

where Co has n rows and nm columns.

The system is controllable if Co has full rank n.

The Matlab code to find out the controllability matrix is “Co = ctrb(sys)”

Observability:

The observability question deals with the question of while not knowing the initial state, whether one can determine the state from the input and output. This is equivalent to the question of whether one can determine the initial state $x(0)$ when given the input $u(t)$, $t \in [0, T]$ and the output $y(t)$, $t \in [0, T]$.

It can also be stated as, a system with an initial state, $x(t_0)$ is observable if and only if the value of the initial state can be determined from the system output $y(t)$ that has been observed through the time interval $t_0 < t < t_f$. If the initial state cannot be so determined, the system is unobservable.

with n columns and np rows.

The model is observable if Ob has full rank n.

The Matlab code to determine the observability matrix is “Ob = obsv(sys)”

$$\text{Ob} = \begin{pmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{pmatrix}$$

SYSTEM RESULTS :

For our developed design we have the transfer function stated as

$$Tf = \frac{19.2s^3 + 80.88s^2 + 274.9s + 468.7}{s^3 + 2.785s^2 + 5.284s + 1.846}$$

By using the Matlab code “[A,B,C,D]=tf2ss(num,den)” we obtain the following state space parameters.

$$A = \begin{bmatrix} -2.7850 & -5.2840 & -1.8460 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

$$C = \begin{bmatrix} 27.4080 & 173.4472 & 433.2568 \end{bmatrix}$$

$$D = 19.200$$

By using the obtained parameters and using the codes of controllability and observability matrix in Matlab we get the respective Co and Ob matrices as

$$Co = \begin{bmatrix} 1 & -2.7850 & 2.4722 \\ 0 & 1 & -2.7850 \\ 0 & 0 & 1 \end{bmatrix} \text{ and}$$

$$Ob = \begin{bmatrix} 27.4080 & 97.1159 & 17.9651 \\ 173.4472 & 288.4329 & -563.7557 \\ 433.2568 & -50.5952 & -179.2760 \end{bmatrix}$$

Further analyzing on the derived results it is found that both the matrices satisfy the given conditions for controllability and observability. Hence we can conclude saying that the system developed is completely observable and controllable.

OBSERVATIONS:

Post fabrication results of the constructed TRMS showed a range of observations in mechanical, electronic and programming backgrounds. The system's unique design and new component set compared to the previous TRMS designs has given a major set of difference in practical TRMS and those observations are listed below.

First and foremost, the design of TRMS which was fabricated gave us a completely different set of dynamics compared to the ones described in our literature survey. To make the study mode informative and qualitative the counter balance provided in the system was in line with the main beam. This gave us a different view of system identification as the system is now more centre of gravity centric. The fabrication phase saw a lot of design changes and we finally came up with the above proposed design.

In earlier designs, the actuator used was usually a brushed DC motor. The brushed motors exhibit more linear and constant slope characteristics. But in the project the team decided to adopt a BLDC motor, as the actuator gives us more aggressive dynamic character, which makes the system more dynamic and the controlling will be more challenging. As speculated the BLDC exhibited very high aggressiveness and thus presented a more dynamic system to model and control.

While performing the parameter estimation, the ARX and ARMAX model had a huge difference when performed on the same system. The ARX model gives us a proper estimate as it considers the error, and the first order system of ARX gave us a perfect match within the first iteration.

Another important observation is the after settling for first order, the system when modeled for higher order ARX algorithm resulted in zero coefficients apart from the first noise model parameter. The ARMAX modeling when performed on the TRMS gave a more generic first order model, whereas the ARMAX is proved to be a more approximate model with noise interference. But if the system increases in ARMAX modeling, the system becomes less tolerant to noise interference.

Recommendations:

The main reason behind choosing this project is to develop a industry based test bench with which a number of control experiments can be performed. This year we have fabricated a new test bench model and performed system identification using ARX and ARMAX methods modeling. Also the project was restricted to black box modeling of TRMS. With this apparatus a more wider range of parameter estimation using different level of certainty (white, grey models) which can sum up to literature in numerous research backgrounds.

REFERENCES:

- [1]Ahmad, S. M., A. J. Chipperfield, and M. O. Tokhi (2000). Dynamic modeling and control of a 2 DOF twin rotor multi-input multi-output system. In Proc. of Conf. of IEEE Industrial Electronics Society, Nagoya, Japan, pp. 1451–1456.
- [2]Bayrak, A., M. H. Salah, N. Nath, and E. Tatlicioglu (2010). Neural network-based nonlinear control design for twin rotor MIMO systems. In Proc. of Int. Symposium on Mechanism and Machine Science, Izmir, Turkey, pp. 172–178.
- [3]Intan Zaurah Mat Darus and Zainul Aman Lokaman(2010).Dynamic Modeling of Twin Rotor Multi System In HorizonTal Motion. Jurnal Mekanikal No.31,17-29
- [4]A Rahideh and M.H.Shaeed(2007) Mathematical dynamic modeling of a twin-rotor Multiple - Input multiple-Output System. Proceedings of Institution of Mechanical Engineers, Part 1:Journal of Systems and Control Engineering.

APPENDIX :

Programs:

MATLAB program for ARMAX modelling:

```
import firstorder;

import secondorder;

import thirdorder;

d = wgn(355,1);

first_t = transpose(firstorder);

second_t = transpose(secondorder);

fourth_t = transpose(fourthorder);

first1 = inv(first_t*firstorder);

second1=inv(second_t*secondorder);

fourth1 = inv(fourth_t*fourthorder);

import = yaw_input;

first2 = first_t*y_first;

second2 = second_t*y_second;

fourth2= fourth_t * y_fourth;

final1 = firstorder*(first1*first2);

final2 = secondorder*(second1*second2);

final4 = fourthorder*(fourth1*fourth2);

err1 = y_first - final1;
```

```

err2 = y_second - final2;

err3 = y_fourth - final4;

first1_t = transpose(firstorder1);

second1_t = transpose(secondorder1);

fourth1_t = transpose(fourthorder1);

first1_1 = firstorder1*first1_t;

second1_1 = secondorder1*second1_t;

fourth1_1 = fourthorder1*fourth1_t;

first1_2 = first1_t * y_first;

second1_2 = second1_t * y_second;

fourth1_2 = fourth1_t * y_fourth;

% final1_1 = first1_1 * first1_2;

% final1_2 = second1_1 * second1_2;

% final1_4 = fourth1_1 * fourth1_2;

firstorder_armax = pinv(firstorder1);

secondorder_armax = pinv(secondorder1);

fourthorder_armax = pinv(fourthorder1);

final1_1 = firstorder_armax * y_first;

final1_2 = secondorder_armax * y_second;

final1_4 = fourthorder_armax * y_fourth;

finalarmax1 = firstorder1 * final1_1;

```

```

finalarmax2 = secondorder1 * final1_2;

finalarmax3 = fourthorder1 * final1_4;

t = [0:1:356];

% plot(t,final1);

% hold on;

% t1 = [0:1:355];

% plot(t1,final2);

% % hold on;

% t2 = [0:1:353];

% plot(t2,final4);

% hold on;

plot(t,y_first);

hold off;

```

ARX modelling MATLAB code :

```

import firstorder;

import secondorder;

import thirdorder;

import yaw_input;

```

```

second1=inv(second_t*secondorder);

fourth1 = inv(fourth_t*fourthorder);

first2 = first_t*y_first;

second2 = second_t*y_second;

final1 = firstorder*(first1*first2);

err1 = y_first - final1;

final2 = secondorder*(second1*second2);

fourth2 = fourth_t * y_fourth;

final4 = fourth1*fourth2;

plot(t,final2);

```

ARDUINO PROGRAMS:

ESC Caliberation program:

```

#include <Servo.h>

Servo S; // create servo object to control a servo

Servo S1;

int pos = 1000;

void setup()

{

    S.attach(6);

    S1.attach(5);

```

```
Serial.begin(9600);
```

```
pos++;
```

```
S.writeMicroseconds(pos);
```

```
delay(1000);
```

```
}
```

```
void loop()
```

```
{
```

```
if (Serial.available())
```

```
{
```

```
char ch = Serial.read();
```

```
if (ch == '+')
```

```
{
```

```
pos = pos + 10;
```

```
}
```

```
if (ch == 'p')
```

```
{
```

```
pos = pos + 1;
```

```
}
```

```
if (ch == 'm')
```

```
{  
  
    pos = pos - 1;  
  
}  
  
    if(ch == '-')  
  
    {  
  
        pos = pos - 10;  
  
    }  
  
    if(ch == '0')  
  
    {  
  
        pos = 1000;  
  
    }  
  
    if(ch == '1')  
  
    {  
  
        pos = 2000;  
  
    }  
  
}  
  
//pos = 1280;  
  
S.writeMicroseconds(pos);  
  
S1.writeMicroseconds(pos);
```

```
Serial.println(pos);

}

/*end of code*/
```

Arduino code for data logging :

```
#include <SD.h>

#include <Servo.h>

#include <SPI.h>

File file;

int CS_PIN = 10;

void initializeSD()

{

    Serial.println("Initializing SD card...");

    pinMode(CS_PIN, OUTPUT);

    if (SD.begin())

    {

        Serial.println("SD card is ready to use.");

    } else

    {
```

```
    Serial.println("SD card initialization failed");

    return;

}

}

int createFile(char filename[])

{

    file = SD.open(filename, FILE_WRITE);

    if (file)

    {

        Serial.println("File created successfully.");

        return 1;

    } else

    {

        Serial.println("Error while creating file.");

        return 0;

    }

}

int writeToFile(char text[])

{
```

```
if (file)

{

    file.println(text);

    Serial.println("Writing to file: ");

    Serial.println(text);

    return 1;

} else

{

    Serial.println("Couldn't write to file");

    return 0;

}}
```

```
void closeFile()
```

```
{

    if (file)

    {

        file.close();

        Serial.println("File closed");

    }

}
```

```

int openFile(char filename[])

{

    file = SD.open(filename);

    if (file)

    {

        Serial.println("File opened with success!");

        return 1;

    } else

    {

        Serial.println("Error opening file...");

        return 0;

    }

}

String readLine()

{

    String received = "";

    char ch;

    while (file.available())

```

```

{

    ch = file.read();

    if(ch == '\n')

    {

        return String(received);

    }

    else {

        received += ch;

    } }

    return "";}

Servo esc1;

Servo esc2;

#include "Wire.h"

#include "I2Cdev.h"

#include "MPU6050_6Axis_MotionApps20.h"

MPU6050 mpu(0x69);

#define LED_PIN 13 // (Arduino is 13, Teensy is 11, Teensy++ is 6)

bool blinkState = false;

bool dmpReady = false; // set true if DMP init was successful

```

```

uint8_t mpuIntStatus; // holds actual interrupt status byte from MPU

uint8_t devStatus;    // return status after each device operation (0 = success, !=0 = error)

uint16_t packetSize;  // expected DMP packet size (default is 42 bytes)

uint16_t fifoCount;    // count of all bytes currently in FIFO

uint8_t fifoBuffer[64]; // FIFO storage buffer

Quaternion q;

VectorFloat gravity;

float euler[3];

float ypr[3];

unsigned long last_read_time;

float    last_x_angle; // These are the filtered angles

float    last_y_angle;

float    last_z_angle;

float    last_gyro_x_angle; // Store the gyro angles to compare drift

float    last_gyro_y_angle;

float    last_gyro_z_angle;


void set_last_read_angle_data(unsigned long time, float x, float y, float z, float x_gyro, float
y_gyro, float z_gyro) {

```

```
last_read_time = time;

last_x_angle = x;

last_y_angle = y;

last_z_angle = z;

last_gyro_x_angle = x_gyro;

last_gyro_y_angle = y_gyro;

last_gyro_z_angle = z_gyro;

}
```

```
inline unsigned long get_last_time() {return last_read_time;}
```

```
inline float get_last_x_angle() {return last_x_angle;}
```

```
inline float get_last_y_angle() {return last_y_angle;}
```

```
inline float get_last_z_angle() {return last_z_angle;}
```

```
inline float get_last_gyro_x_angle() {return last_gyro_x_angle;}
```

```
inline float get_last_gyro_y_angle() {return last_gyro_y_angle;}
```

```
inline float get_last_gyro_z_angle() {return last_gyro_z_angle;}
```

```
float base_x_gyro = 0;
```

```
float base_y_gyro = 0;
```

```
float base_z_gyro = 0;
```

```

float  base_x_accel = 0;

float  base_y_accel = 0;

float  base_z_accel = 0;

float  GYRO_FACTOR;

float  ACCEL_FACTOR;

int16_t ax, ay, az;

int16_t gx, gy, gz;

char dataOut[256];

volatile bool mpuInterrupt = false;    // indicates whether MPU interrupt pin has gone high

void dmpDataReady() {

    mpuInterrupt = true;

} void calibrate_sensors() {

    int    num_readings = 10;

    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

    for (int i = 0; i < num_readings; i++) {

        mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

        base_x_gyro += gx;

        base_y_gyro += gy;

        base_z_gyro += gz;

```

```
    base_x_accel += ax;

    base_y_accel += ay;

    base_z_accel += az;

}

base_x_gyro /= num_readings;

base_y_gyro /= num_readings;

base_z_gyro /= num_readings;

base_x_accel /= num_readings;

base_y_accel /= num_readings;

base_z_accel /= num_readings;

}

int op =A5,oop=6;

void setup() {

    pinMode(op,OUTPUT);

    pinMode(oop,OUTPUT);

    Serial.begin(9600);

    esc1.attach(8);

    esc2.attach(9);
```

```
Wire.begin();
```

```
Serial.begin(9600);
```

```
while (!Serial); // wait for Leonardo enumeration, others continue immediately
```

```
Serial.println(F("Initializing I2C devices..."));
```

```
mpu.initialize();
```

```
// verify connection
```

```
Serial.println(F("Testing device connections..."));
```

```
Serial.println(mpu.testConnection() ? F("MPU6050 connection successful") : F("MPU6050  
connection failed"));
```

```
// load and configure the DMP
```

```
Serial.println(F("Initializing DMP..."));
```

```
devStatus = mpu.dmpInitialize();
```

```
// make sure it worked (returns 0 if so)
```

```
if (devStatus == 0) {
```

```
    Serial.println(F("Enabling DMP..."));
```

```
    mpu.setDMPEnabled(true);
```

```
    Serial.println(F("Enabling interrupt detection (Arduino external interrupt 0)..."));
```

```
    attachInterrupt(0, dmpDataReady, RISING);
```

```
    mpuIntStatus = mpu.getIntStatus();
```

```
    Serial.println(F("DMP ready! Waiting for first interrupt..."));
```

```
    dmpReady = true;
```

```
    uint8_t FS_SEL = 0;
```

```
uint8_t READ_FS_SEL = mpu.getFullScaleGyroRange();
```

```
Serial.print("FS_SEL = ");
```

```
Serial.println(READ_FS_SEL);
```

```
GYRO_FACTOR = 131.0/(FS_SEL + 1);
```

```
uint8_t READ_AFS_SEL = mpu.getFullScaleAccelRange();
```

```
Serial.print("AFS_SEL = ");
```

```
Serial.println(READ_AFS_SEL);
```

```
packetSize = mpu.dmpGetFIFOPageSize();
```

```
} else {
```

```
Serial.print(F("DMP Initialization failed (code "));
```

```
Serial.print(devStatus);
```

```
Serial.println(F(""));
```

```
}
```

```
// configure LED for output

pinMode(LED_PIN, OUTPUT);


// get calibration values for sensors

calibrate_sensors();

set_last_read_angle_data(millis(), 0, 0, 0, 0, 0, 0);

esc1.write(0);

esc2.write(0);

initializeSD();

delay(500);

}

bool f=0,g=0;


int i = 0;

void loop() {

    const float RADIANS_TO_DEGREES = 57.2958; //180/3.14159


    if (!dmpReady) return;
```

```
unsigned long t_now = millis();
```

```
while (!mpuInterrupt && fifoCount < packetSize) {
```

```
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
```

```
    // Get time of last raw data read
```

```
    unsigned long t_now = millis();
```

```
    // Remove offsets and scale gyro data
```

```
    float gyro_x = (gx - base_x_gyro)/GYRO_FACTOR;
```

```
    float gyro_y = (gy - base_y_gyro)/GYRO_FACTOR;
```

```
    float gyro_z = (gz - base_z_gyro)/GYRO_FACTOR;
```

```
    float accel_x = ax; // - base_x_accel;
```

```
    float accel_y = ay; // - base_y_accel;
```

```
    float accel_z = az; // - base_z_accel;
```

```
float accel_angle_y = atan(-1*accel_x/sqrt(pow(accel_y,2) +  
pow(accel_z,2)))*RADIANS_TO_DEGREES;
```

```
float accel_angle_x = atan(accel_y/sqrt(pow(accel_x,2) +  
pow(accel_z,2)))*RADIANS_TO_DEGREES;
```

```
float accel_angle_z = 0;
```

```
// Compute the (filtered) gyro angles
```

```
float dt =(t_now - get_last_time())/1000.0;
```

```
float gyro_angle_x = gyro_x*dt + get_last_x_angle();
```

```
float gyro_angle_y = gyro_y*dt + get_last_y_angle();
```

```
float gyro_angle_z = gyro_z*dt + get_last_z_angle();
```

```
// Compute the drifting gyro angles
```

```
float unfiltered_gyro_angle_x = gyro_x*dt + get_last_gyro_x_angle();
```

```
float unfiltered_gyro_angle_y = gyro_y*dt + get_last_gyro_y_angle();
```

```
float unfiltered_gyro_angle_z = gyro_z*dt + get_last_gyro_z_angle();
```

```
const float alpha = 0.96;
```

```
float angle_x = alpha*gyro_angle_x + (1.0 - alpha)*accel_angle_x;
```

```
float angle_y = alpha*gyro_angle_y + (1.0 - alpha)*accel_angle_y;
```

```
float angle_z = gyro_angle_z;
```

```
    set_last_read_angle_data(t_now, angle_x, angle_y, angle_z, unfiltered_gyro_angle_x,  
unfiltered_gyro_angle_y, unfiltered_gyro_angle_z);
```

```
}
```

```
mpuInterrupt = false;
```

```
mpuIntStatus = mpu.getIntStatus();
```

```
fifoCount = mpu.getFIFOCount();
```

```
if ((mpuIntStatus & 0x10) || fifoCount == 1024) {
```

```
    // reset so we can continue cleanly
```

```
    mpu.resetFIFO();
```

```
    Serial.println(F("FIFO overflow!"));
```

```
} else if (mpuIntStatus & 0x02) {
```

```
    while (fifoCount < packetSize) fifoCount = mpu.getFIFOCount();
```

```
    mpu.getFIFOBytes(fifoBuffer, packetSize);
```

```
    fifoCount -= packetSize;
```

```
    mpu.dmpGetQuaternion(&q, fifoBuffer);
```

```
    mpu.dmpGetGravity(&gravity, &q);
```

```
    mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);
```

```
    Serial.print("CMP:");
```

```
Serial.print(get_last_x_angle(), 2);

Serial.print(":");

Serial.print(get_last_y_angle(), 2);

Serial.print(":");

Serial.println(-get_last_z_angle(), 2);

Serial.print("DMP:");

Serial.print(ypr[2]*RADIANS_TO_DEGREES, 2);

Serial.print(":");

Serial.print(-ypr[1]*RADIANS_TO_DEGREES, 2);

Serial.print(":");

Serial.println(ypr[0]*RADIANS_TO_DEGREES, 2);

int a = ypr[2]*RADIANS_TO_DEGREES;

int b = ypr[0]*RADIANS_TO_DEGREES;

char c[4],d[4] ;

itoa(a,c,10);

itoa(b,d,10);

int gg = random(10,80);

int hh = random(10,80);

esc1.write(gg);
```

```
    esc2.write(hh);

    f = !f;

    g = !g;

    digitalWrite(op,f);

    digitalWrite(oop,g);

    File dataFile = SD.open("readings.txt", FILE_WRITE);

    if (dataFile) {

dataFile.print(i);

dataFile.print(" ");

dataFile.print(gg);

dataFile.print(" ");

dataFile.print(hh);

dataFile.print(" ");

dataFile.print(c);

dataFile.print("    ");

dataFile.println(d);

dataFile.close();}

    i++;

    blinkState = !blinkState;
```

```
digitalWrite(LED_PIN, blinkState);
```

```
delay(500);}}
```